MissingPieces: Automated Fashion Attribute Recognition from User-Generated Images

A Computer Vision Pipeline for Garment Segmentation, Color Extraction, and Clothing Category Classification

Course: Introduction to Computer Vision
Project Area: Fashion Attribute Recognition and Automated Catalog Ingestion
Dataset: agrigorev/clothing-dataset-full — Kaggle
Main Techniques: GrabCut, HOG, LBP, SVM, K-Means, ResNet18 Transfer Learning

1. Problem Statement

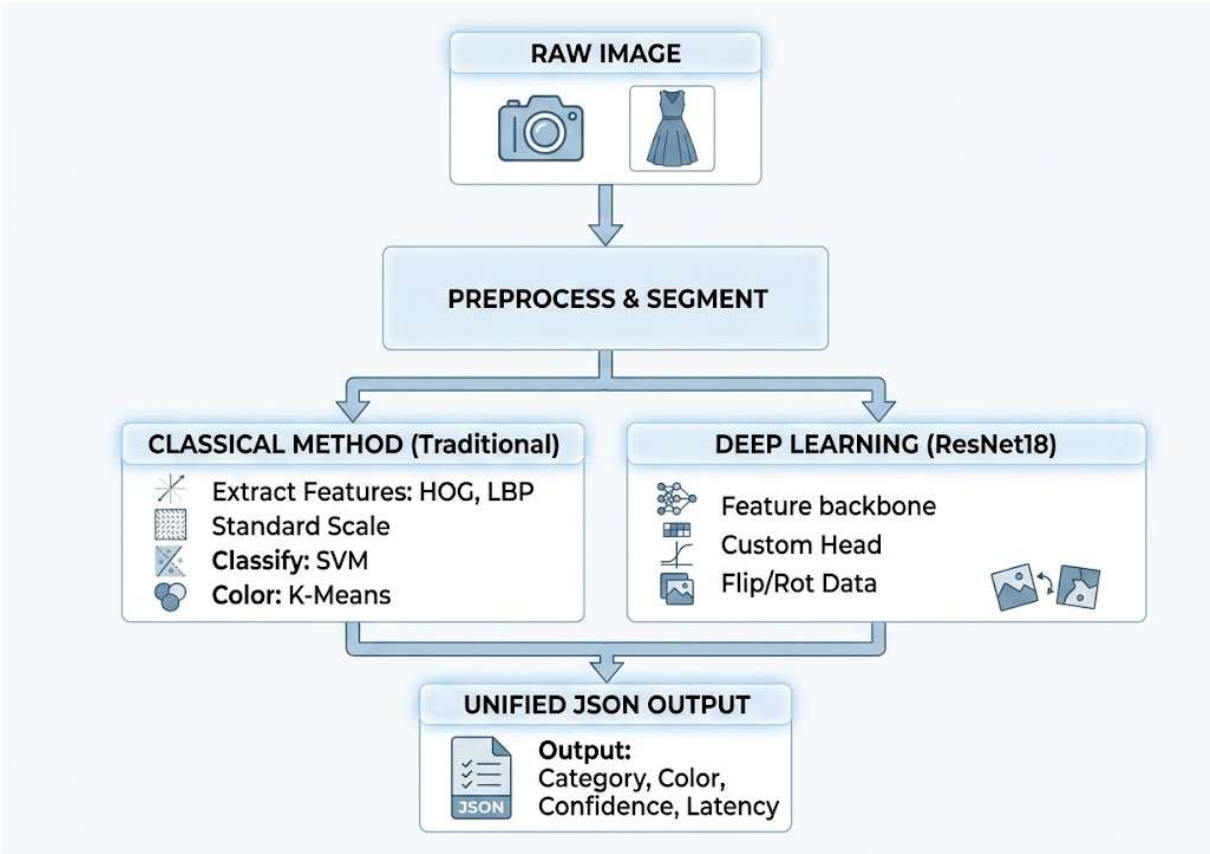
Digital wardrobe applications and online second-hand marketplaces rely on accurate and consistent product tags—such as garment category and dominant color—to make items easy to search, compare, and filter. In practice, these attributes are often entered manually by users, which makes the process repetitive and time-consuming, and can introduce subjective inconsistencies such as classifying a light jacket as a shirt or describing the same color in different ways.
The objective of this project is to build an automated computer vision pipeline that starts from a single raw photograph of a clothing item and produces three practical outputs:

1. An isolated foreground representation of the garment.
2. The primary dominant color in standard HEX and RGB formats.
3. The predicted clothing category.

The solution is tested on unconstrained images that reflect realistic user conditions, including clothes photographed on wooden floors, beds, or rugs and under different types of home lighting. This setting makes it possible to compare a traditional feature-engineering approach with a deep transfer-learning model in a scenario closer to everyday use.

The work was developed with the goal of keeping the pipeline understandable and reproducible. For this reason, I first tested a simpler method and then compared it with a more advanced model, so that the improvement could be evaluated step by step instead of treating deep learning as a black box.

2. Methodology



2.1 Dataset Preparation and Cleaning

The pipeline is based on the Kaggle clothing-dataset-full repository, which initially contains 5,403 raw entries. Before training and evaluation, the dataset is cleaned and consolidated through the following steps:

- Removing entries marked with ambiguous labels (Not sure) or missing annotations.
- Verifying disk file availability, leaving 5,175 verified images.
- Filtering the dataset to the top 10 most represented classes (4,514 images): T-Shirt, Longsleeve, Pants, Shoes, Shirt, Dress, Outwear, Shorts, Hat, and Skirt.

2.2 Preprocessing, Segmentation, and Mask Cleanup

Before classification and color extraction, each image needs to be prepared in a consistent way. The original photos can have different sizes, lighting conditions, and backgrounds, so this step helps the pipeline focus mainly on the clothing item instead of the surrounding environment.

1. **Resizing:** All images are resized to 224*224 pixels and converted to RGB format. This gives the model inputs with the same size and color representation, which is necessary to process the images in a uniform way.

2. **Garment segmentation:** I used GrabCut to separate the main clothing item from the background. The algorithm starts from a rectangle placed slightly inside the image borders and then estimates which pixels probably belong to the garment and which ones belong to the background.
3. **Mask cleanup:** After segmentation, the resulting mask is cleaned to remove small errors, such as isolated background pixels or small holes inside the garment area. This makes the extracted garment region more stable and helps avoid mistakes during color extraction and classification.

2.3 Classical Baseline: HOG + LBP + SVM

As a first approach, I implemented a classical machine learning baseline. The purpose of this baseline was not only to obtain a result, but also to have a simpler and more explainable comparison point before moving to a deep learning model.

- **HOG:** This feature descriptor was used to capture the general shape and contour of each garment, which is useful because many clothing categories differ mainly by their silhouette.
- **LBP:** This descriptor was used to include basic texture information, for example differences between smooth surfaces, patterned fabrics, or stronger visual details.
- **SVM classifier:** The extracted visual features were then used to train a Support Vector Machine. I used a balanced subset of the dataset for this part, because the complete feature extraction process was slower and heavier on the CPU.
- **Color extraction:** The dominant color was estimated by grouping the garment pixels into a small number of color clusters and selecting the most representative one.

2.4 Deep Learning: Transfer Learning with ResNet18

After the classical baseline, I tested a deep learning approach based on transfer learning. This choice was motivated by the fact that training a convolutional neural network from zero would require much more data and computational resources, while a pre-trained ResNet18 already contains useful visual patterns learned from a large image dataset.

- **Model backbone:** The initial convolutional layers were kept frozen, so the model could reuse general visual features such as edges, shapes, and simple patterns.
- **Final classification layers:** The original final layer was replaced with a smaller custom classifier adapted to the 10 clothing categories of this project.
- **Training setup:** The model was trained for 5 epochs using common image augmentation techniques such as horizontal flipping, small rotations, and brightness or contrast variations. These transformations helped make the model less dependent on a specific image condition.

3. Experimental Results

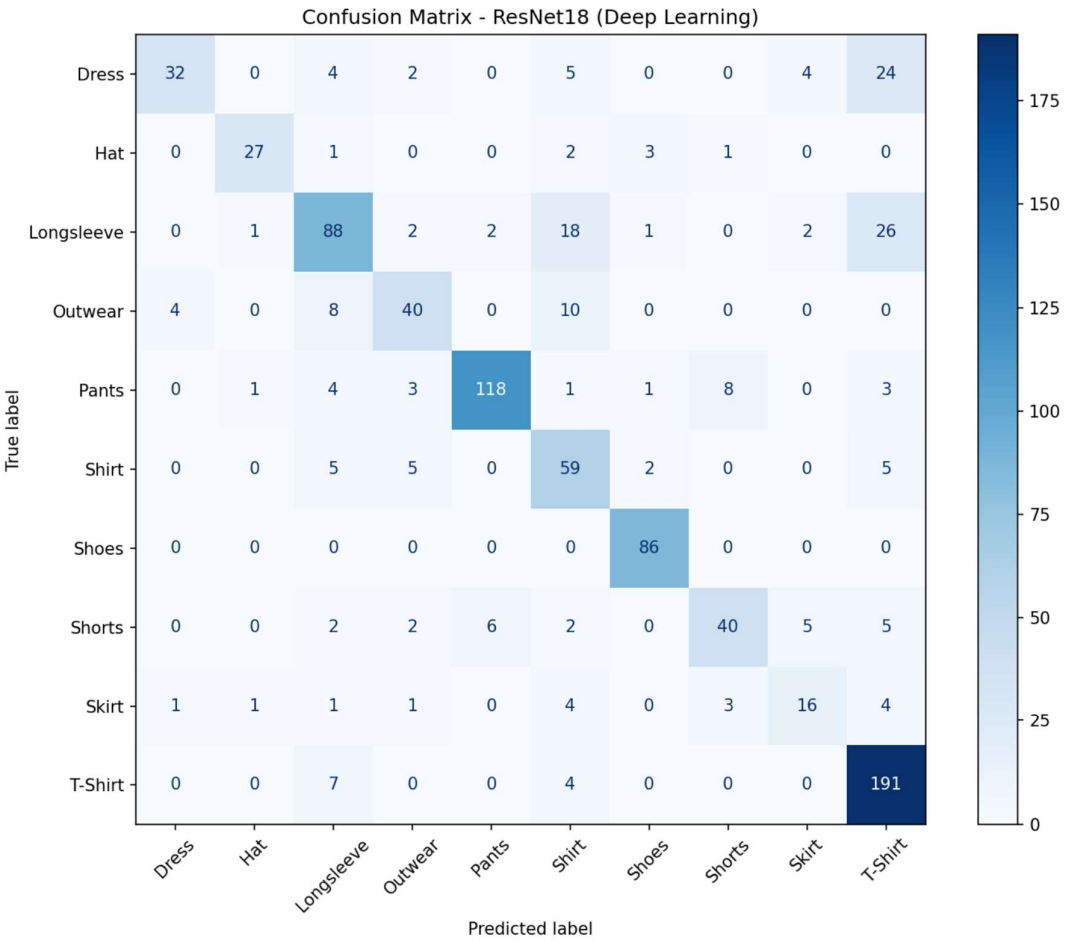
Both pipelines were evaluated on stratified validation splits, keeping the comparison consistent with the available data for each approach: 903 test images for ResNet18 and 300 test images for the balanced classical split.
I considered accuracy together with the macro F1-score because the dataset is not perfectly balanced. Accuracy gives a general overview of the result, while macro F1 is more useful to understand whether the model performs reasonably well across all classes, including the less represented ones.

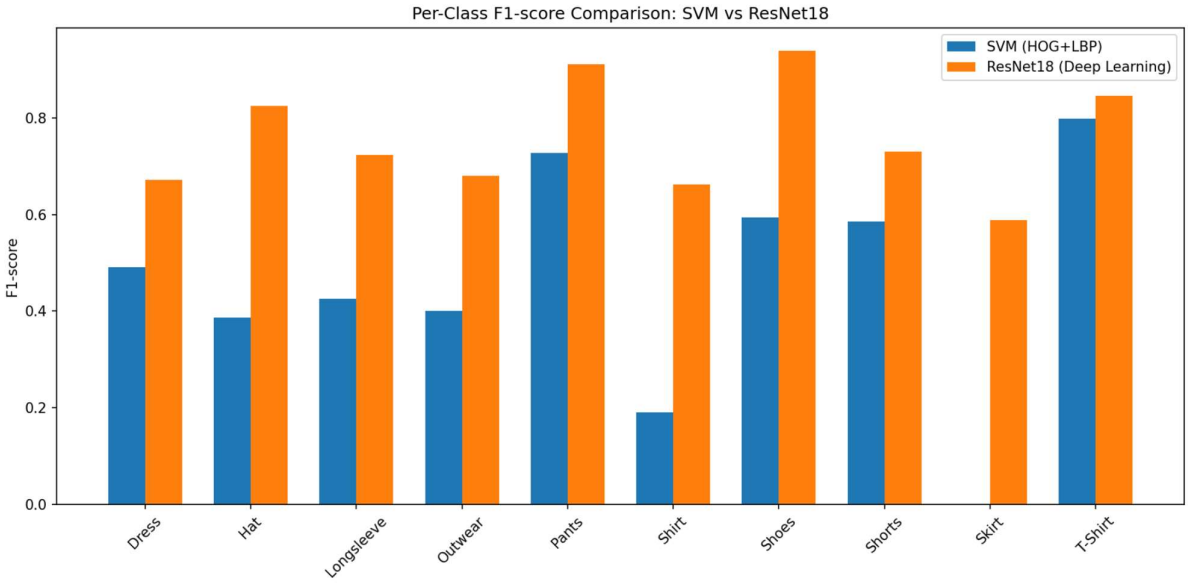
Overall Performance & Latency Comparison

Model	Accuracy	Macro F1	Classifier Latency (ms/img)	Full Pipeline Latency (ms/img)
Classical (HOG + LBP + SVM)	60.00%	0.46	~6.85 ms	~221.6 ms (CPU)
Deep Learning (ResNet18)	**78.85%**	**0.76**	~107.5 ms	**~107.5 ms (GPU)**

Per-Class F1-Score Breakdown

Class	Classical SVM F1	ResNet18 F1	Support (ResNet Val)
Dress	0.491	0.672	71
Hat	0.387	0.825	34
Longsleeve	0.426	0.723	140
Outwear	0.400	0.680	62
Pants	0.727	0.911	139
Shirt	0.190	0.662	71
Shoes	0.594	0.939	86
Shorts	0.586	0.730	60
Skirt	0.000	0.588	31
T-Shirt	0.798	0.845	209





Main observations:

- ResNet18 clearly outperforms the classical approach across all categories. This result was expected, but the difference is still useful to observe because it shows how much learned features can help when images have different backgrounds, lighting conditions, and garment positions.

- Although SVM inference alone is very fast, the CPU cost of extracting HOG and LBP features makes the full classical pipeline slower than the GPU-based deep learning pipeline when measured end to end.
- From a learning perspective, the comparison was useful because it showed that a simpler method can still produce acceptable results, but it becomes limited when the visual differences between classes are subtle or when the background is not controlled.
- The table also shows that the model performs better on categories with a clear visual structure, such as Shoes and Pants, while it struggles more when the shape of the garment changes a lot from one image to another.

4. Failure Analysis

The error analysis shows that most mistakes are understandable from a visual point of view. Some items are genuinely difficult to distinguish, especially when they are laid flat, partially folded, or photographed under non-ideal lighting conditions.

1. **T-Shirt overprediction:**
   The model sometimes predicted T-Shirt when the image was uncertain. This is likely related to the fact that T-Shirts are the most frequent class in the dataset, so the model sees many examples of them during training.
2. **Shirt and Longsleeve confusion:**
   These two classes were sometimes confused because they can have very similar shapes. In some images, details such as buttons, collars, or cuffs are not clearly visible, so the model has less information to separate the two categories.
3. **Skirt recognition:**
   Skirt was one of the most difficult classes. This may be caused by the small number of examples and by the fact that skirts can take very different shapes depending on how they are positioned in the image.
4. **Background and lighting:**
   Some errors may also be influenced by shadows, similar background colors, or images where the garment is not perfectly centered. This is a common limitation when working with real user images instead of controlled studio photos.

5. Ethical Considerations

- **Representation Bias:** The dataset is more representative of common Western everyday apparel, such as T-Shirts and Jeans, than of less frequent styles. In a production context, underrepresented garments, traditional attire, or niche categories

would likely require additional balanced training data to avoid systematically higher error rates.

- **User Privacy:** Images captured at home may include parts of private environments, such as floors, bedrooms, or household objects. For this reason, the pipeline applies foreground segmentation and discards background pixels before attribute extraction, so that only garment-related information is carried forward into the catalog output.
- **Automation Risk:** Automated metadata assignment should support the user rather than replace final human validation. Before an item is published, the interface should allow users to confirm or correct the predicted attributes, reducing the risk of inaccurate listings such as a skirt being published as shorts.

For these reasons, I would not consider the system as a fully autonomous decision-maker. A more realistic use case is an assistant tool that suggests labels to the user, who can then accept or correct them before publishing the item.
Another point to consider is transparency. Even if the system returns only a predicted label, it is important that users understand that the result is probabilistic and may need correction. Showing a confidence score can help communicate this uncertainty in a simple way.

6. System Output Example

At the end of the workflow, the inference function returns the predicted class, the dominant color, and the main execution metrics in a standardized JSON response. This format makes the output easy to integrate into downstream cataloging or ingestion systems.
JSON


```
{
  "predicted_category": "Shoes",
  "confidence": 0.4375,
  "dominant_color_hex": "#d0c7bf",
  "dominant_color_rgb": [208, 199, 191],
  "avg_inference_latency_ms": 107.53
}
```


In this example, the confidence value is not extremely high. This is useful to notice because it means that, even when the predicted class is reasonable, the system should still allow the user to review the result before using it in a real catalog.

7. Conclusion and Future Work

This project showed that computer vision can be effectively applied to a practical fashion cataloging scenario. The classical approach was useful to understand the role of handcrafted

visual features, but its performance was limited, especially for visually similar or less represented classes. The ResNet18 transfer-learning model achieved better results overall and proved to be more suitable for this type of image classification task.

In the context of the Missing Piece application, this model could become a useful support component during the item upload process. Instead of asking the user to manually select all attributes, the system could automatically suggest the garment category and dominant color from the uploaded image. The user would still keep control of the final decision by reviewing, confirming, or correcting the suggested values before saving the item.
At the same time, the results also highlight some areas that should be improved before using the model in a real application. A larger and more balanced dataset would help reduce errors on less represented classes, such as Skirt and Shirt. Better segmentation methods could also make the system more robust when images contain complex backgrounds, shadows, or similar colors between the garment and the surrounding environment. Finally, additional fine-tuning of the neural network could improve the model accuracy without changing the general structure of the pipeline.

Overall, the work represents a solid starting point for completing Missing Piece with an automated but still user-controlled recognition feature. From a learning perspective, it also helped me understand both the potential and the limitations of combining traditional computer vision techniques with modern deep learning models.